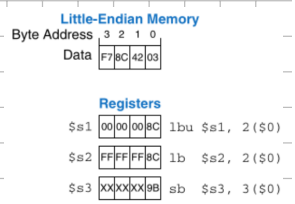
Character: char in C is 8-bit → it uses the 8-bit ASCII

→ the lbu (load byte unsigned), lb, sb ins. exist for bytes like ASCII encodings → useful for I/O handling
(lbu: zero-extends byte; lb: sign-extends byte)



→ in the example, mem. addr. byte at addr. 2 loaded into LSB of $1, zero-extended
→ $2 lb loads sign-extended byte, $3 call stores into byte 3 of mem.

Procedure Calls: yes, you can call methods in MIPS assembly

→ the caller puts up to 4 arguments into $a0-$a3 before making call, callee places return values into $v0-$v1

→ callee must know where to return → caller stores return address in $ra at the same time it jumps to callee using jal

→ callee cannot overwrite caller's arch. state or memory → leave $s0-$s7, $ra, and the stack intact

→ jal used to call, jr to return → jal stores address of next ins. in $ra, jumps to target ins.

→ when 64-bit value is returned, we need both return regs., $v0, $v1

High-Level Code

```
int main ()
{
  int y;
  ...
  y = diffofsums (2, 3, 4, 5);
  ...
}

int diffofsums (int f, int g, int h, int i)
{
  int result;
  result = (f + g) - (h + i);
  return result;
}
```

MIPS Assembly Code

```
# $s0 = y
main:
  ...
  addi $a0, $0, 2   # argument 0 = 2
  addi $a1, $0, 3   # argument 1 = 3
  addi $a2, $0, 4   # argument 2 = 4
  addi $a3, $0, 5   # argument 3 = 5
  jal diffofsums    # call procedure
  add $s0, $v0, $0  # y = returned value
  ...
# $s0 = result
diffofsums:
  add $t0, $a0, $a1 # $t0 = f + g
  add $t1, $a2, $a3 # $t1 = h + i
  sub $s0, $t0, $t1 # result = (f + g) - (h + i)
  add $v0, $s0, $0  # put return value in $v0
  jr  $ra           # return to caller
```

The Stack (you need to know this concept beyond DDCA!)

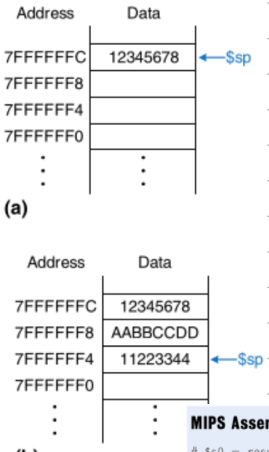
→ memory that saves local vars. in a procedure

→ expands when more vars. are needed, contracts when less are needed

→ it is a LIFO queue (like the data structure Stack in Java) where the last item pushed is the first to be popped

→ top of the stack: most recently allocated space → MIPS stack grows down in memory, expands to lower mem. addresses

→ the stack pointer $sp is a special reg. that points to the top of the stack → pointers point to mem. addresses



→ stack is used to save and restore regs. used by a procedure

→ a procedure should not modify regs. except for return value

→ registers are saved on the stack before modification, then restored from stack before return

→ steps:

→ 1. make space on stack to store reg. values, 2. store those on stack

→ 3. execute procedure using regs., 4. restore original values from stack, 5. deallocate stack space

MIPS Assembly Code (Fixed to align with stack)

```
# $s0 = result
diffofsums:
  addi $sp, $sp, -12  # make space on stack to store three registers
  sw   $s0, 8($sp)    # save $s0 on stack   ] stack frame, stack space a
  sw   $t0, 4($sp)    # save $t0 on stack   ] procedure allocates for itself
  sw   $t1, 0($sp)    # save $t1 on stack   ]
  add  $t0, $a0, $a1  # $t0 = f + g
  add  $t1, $a2, $a3  # $t1 = h + i
  sub  $s0, $t0, $t1  # result = (f + g) - (h + i)
  add  $v0, $s0, $0   # put return value in $v0
  lw   $t1, 0($sp)    # restore $t1 from stack
  lw   $t0, 4($sp)    # restore $t0 from stack
  lw   $s0, 8($sp)    # restore $s0 from stack
  addi $sp, $sp, 12   # deallocate stack space
  jr   $ra            # return to caller
```

Figure 6.24 The stack

Figure 6.25 The stack (a) before, (b) during, and (c) after diffofsums procedure call

MIPS Assembly Code

```
# $s0 = result
diffofsums:
  addi $sp, $sp, -4   # make space on stack to store one register
  sw   $s0, 0($sp)    # save $s0 on stack
  add  $t0, $a0, $a1  # $t0 = f + g
  add  $t1, $a2, $a3  # $t1 = h + i
  sub  $s0, $t0, $t1  # result = (f + g) - (h + i)
  add  $v0, $s0, $0   # put return value in $v0
  lw   $s0, 0($sp)    # restore $s0 from stack
  addi $sp, $sp, 4    # deallocate stack space
  jr   $ra            # return to caller
```

Preserved Registers: $s regs. are among preserved, $t among nonpreserved

→ procedure must save/restore preserved regs. using stack, free to modify nonpreserved

→ preserved regs. are callee-save, nonpreserved caller-save (caller has to save and restore)

Recursion:

→ procedure that does not call others → leaf procedure, procedure that does not calls others → nonleaf procedures

Table 6.3 Preserved and nonpreserved registers

Preserved	Nonpreserved
Saved registers: $s0-$s7	Temporary registers: $t0-$t9
Return address: $ra	Argument registers: $a0-$a3
Stack pointer: $sp	Return value registers: $v0-$v1
Stack above the stack pointer	Stack below the stack pointer

→ the latter may have to save nonpreserved regs. on stack before they call next proc., then restore after

→ callee saves $f0-$f9 and $a0-$a3, callee saves $s0-$s7, $ra

Example: factorial:

High-Level Code

```
int factorial (int n) {


  if (n <= 1)
      return 1;


  else
    return (n * factorial (n-1));
}
```

MIPS Assembly Code

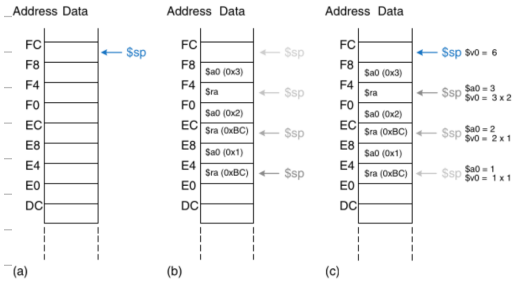
```
0x90  factorial:  addi  $sp, $sp, -8    # make room on stack
0x94              sw    $a0, 4($sp)     # store $a0
0x98              sw    $ra, 0($sp)     # store $ra
0x9C              addi  $t0, $0, 2      # $t0 = 2
0xA0              slt   $t0, $a0, $t0   # a <= 1 ?
0xA4              beq   $t0, $0, else   # no: goto else
0xA8              addi  $v0, $0, 1      # yes: return 1
0xAC              addi  $sp, $sp, 8     # restore $sp
0xB0              jr    $ra             # return
0xB4        else: addi  $a0, $a0, -1    # n = n - 1
0xB8              jal   factorial       # recursive call
0xBC              lw    $ra, 0($sp)     # restore $ra
0xC0              lw    $a0, 4($sp)     # restore $a0
0xC4              addi  $sp, $sp, 8     # restore $sp
0xC8              mul   $v0, $a0, $v0   # n * factorial (n-1)
0xCC              jr    $ra             # return
```

→ factorial is recursive → nonleaf, calls itself

→ factorial saves $a0 and $ra on stack (might modify)

→ checks if n<2 → yes: return 1 in $v0, restore stack pointer, return to caller → doesn't have to reload $ra, $a0 (not modified)

→ if n>1, factorial (n-1) is called → n ($a0) and $ra are restored, multiplies, then returns



→ stack frame extends by 2 words each rec. call

Addressing Modes in MIPS:

→ register-only → all R-types do this → regs. for all sources, destinations

→ immediate → 16-bit immediate used with registers (addi, lui for example)

→ base addressing → mem. access ins. do this (lw, sw) → base + offset from LC-3, find target in mem. by adding base, offset

→ PC-relative: conditional branches do this to specify next PC value if branch taken

→ signed offset in 16-bit imm field added to PC to get new PC

→ pseudo-direct: done by j, jal → 26-bit immediate, direct would use 32-bits → how to get correct PC?

→ 2 LSB's of JTA (jump target addr), $JTA_{1:0}$, should always be 0 → ins. are word-aligned, $JTA_{27:2}$ taken from addr, four MSB's, $JTA_{31:28}$, taken from MSB's of PC+4

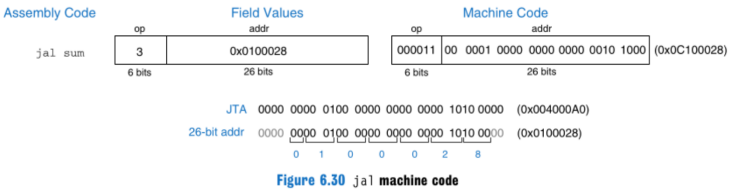


Figure 6.30 `jal` **machine code**

Pseudoinstructions:

→ instructions available to the assembly programmer that run different ins. in reality

Pseudoinstruction	MIPS Instructions
`li $s0, 0x1234AA77`	`lui $s0, 0x1234` `ori $s0, 0xAA77`
`mul $s0, $s1, $s2`	`mult $s1, $s2` `mflo $s0`
`clear $t0`	`add $t0, $0, $0`
`move $s1, $s2`	`add $s2, $s1, $0`
`nop`	`sll $0, $0, 0`

Exceptions:

→ unscheduled procedure call to exception handler

→ caused by hardware → interrupt from I/O
software → trap vectors, real exception

→ how a processor handles them:

→ record cause of exception, jump to exception handler at specific ins. address
→ return to program

→ exception registers are not in the RF
→ Cause records cause of exception
→ EPC records the PC at time of exception

→ Both are in Coprocessor 0 → MIPS processor control for interrupts and exceptions

→ the ins. mfc0 $t0, EPC moves EPC contents into $t0

Exception Causes

Exception	Cause
Hardware Interrupt	0x00000000
System Call	0x00000020
Breakpoint / Divide by 0	0x00000024
Undefined Instruction	0x00000028
Arithmetic Overflow	0x00000030

→ Exception Handler:

→ Saves current registers to stack, reads Cause (mfc0 $t0, Cause), handles exception, restores regs., returns to prog. (mfc0 $k0, EPC → jr $k0)

→ Back to the exam: Prof. Mutlu has not ever made a direct MIPS segment, but it is a key part of pipelining questions (like F525) → we will discuss a question once we do pipelining

→ He does ask Verilog questions:

4.1 What Does This Code Do? [30 points] (F522)

Analyze the following Verilog module and answer the question.

```
1  module mystery_module (clk, en, in1, in2, out);
2
3    input clk, en;
4    input[63:0] in1;
5    input[7:0] in2;
6    output reg[10:0] out = 0;
7
8    reg[2:0] var1 = 0;
9
10   always @(posedge clk) begin
11     out <= out;
12     if (en & (var1 == 0)) begin
13       var1 <= var1 + 1'b1;
14
15       if (in2[var1])
16         out <= 11'd0 + in1[var1*8 +: 8];
17       else
18         out <= 11'd0 - in1[var1*8 +: 8];
19     end
20
21     if (var1 != 0) begin
22       var1 <= var1 + 1'b1;
23
24       if (in2[var1])
25         out <= out + in1[var1*8 +: 8];
26       else
27         out <= out - in1[var1*8 +: 8];
28     end
29   end
30
31 endmodule
```

en = 0 at posedge 0, it equals one at posedge 1, never runs after that

indexed part-select: first term: bit offset, second term: width → example: in1[8+:8] selects in1[15:8]

var1 == 0 at posedge 0

Assume that the inputs in1 and in2 *always* have the following values:

in1 = 64'h0807060504030201

in2 = 8'b11111111

What **unsigned decimal** values does the out signal get in the following waveform diagram? Fill in the gray boxes with an out value for each clk cycle. Briefly explain your answer.

Strategy for solving these:

0.: read the Verilog section if you haven't

1.: look at the blocks in the code and their conditions

2.: go through the code step by step for the relevant inputs and compute results

Here: for posedge 0, 0 is the output (trivial), save for 1

posedge 2: (first if block) var1 set to 1 → line 15 still checks in2[0] (old value)
→ out set to in1[7:0] = 1
second if block is skipped because at evaluation time, var1 == 0

→ in nonblocking assignments, updates are scheduled to the end of the clock edge

→ from posedge 3 on, only the second block is run

all outputs processed → out stays the same

→ so what does the module do? it processes in1 and in2 using var1 as a variable to iterate over in1 8 bits at a time and in2 1 bit at a time → var1 == 0 → 8 LSB's of in1 are added or subtracted to out depending on in2[0], then, we increment var1 every cycle and continue this

Now, we move on to building a CPU and microarchitecture → ISA vs Microarchitecture is free points on the exam: (March)

→ Microarchitecture is the implementation of an ISA under design constraints and goals, including:

→ Pipelining, in order vs. out of order execution, memory access scheduling, speculative execution, superscalar processing, clock gating, caching (levels, size, associativity, replacement), prefetching, voltage/frequency, error correction

design choices

→ in MIPS, the arch. state consists of the PC and RF (necessary)

→ some march. contain nonarch. state (simplify logic / improve performance)

→ our march uses a limited subset of MIPS ins.

→ it consists of the datapath and control → datapath contains mem., RF, ALUs, muxes (operates on words), control unit sets

control signals for datapath elements

→ we start with a simple single-cycle design → control signals are generated in same clock cycle as the one where data signals are being operated on → serialized processing (everything related to ins. happens in 1 cycle)

→ start with HW containing state elements → we separated ins. and data memories → will add CL to connect later on

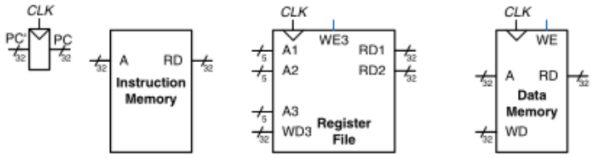


Figure 7.1 State elements of MIPS processor

→ PC is just a 32-bit reg., ins. mem. has one read port

→ RF has 2 read ports, 1 write port, WE, CLK

→ data mem. has one read/write port, WE, CLK

→ memories and RF read combinationally → if addr. changes, new data appears at RD after a propagation delay w/o clock

→ they are only written to at rising edge → state only changes at clock edge

→ address, data, WE must be set up t_{setup} before CLK edge and stay stable for t_{hold} after

→ microprocessor is a sync. seq. circuit

Performance Analysis: (crucial free points on the exam if you know the math)

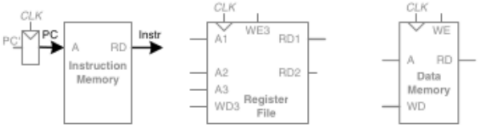
→ execution time of one ins.: {CPI} × {T_c} (clock cycle time, determined by crit. path), of a program: {#instructions} × {avg. CPI} × {T_c}

{cycles per ins.}

→ single-cycle performance: CPI = 1, T_c → long, multi-cycle: CPI: depends on ins., avg. hopefully small, T_c → short

→ upside of multi-cycle: we have two factors to optimize for

Single-cycle datapath: → recall the ins. cycle from the LC-3



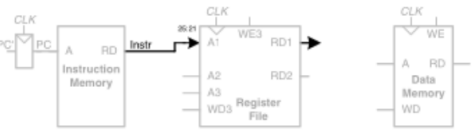
→ ins. fetch: read ins. from ins. mem., PC is connected to address

→ now, let us implement lw: read source reg. ($Instr_{25:21}$) into RD1, sign-extend imm. offset ($Instr_{15:0}$)

important: copy MSB of input into 32b-value

→ $SignImm_{15:0} = Instr_{15:0}$, $SignImm_{31:16} = Instr_{15}$

→ now, compute the mem. address to load by adding base + offset → ALU



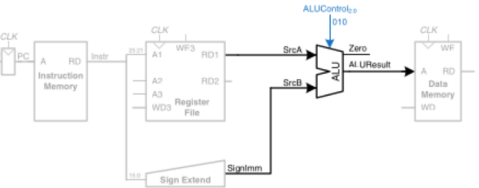


Figure 7.5 Compute memory address

→ ALU adds SrcA from RF, SrcB from sign-ext. imm.

→ ALUControl specifies op. → here 010 (add)

→ Zero flag for if ALUResult == 0

→ ALUResult sent to data memory

→ data from data memory is read onto ReadData bus, sent back to RF (specifically, $Instr_{20:16}$)

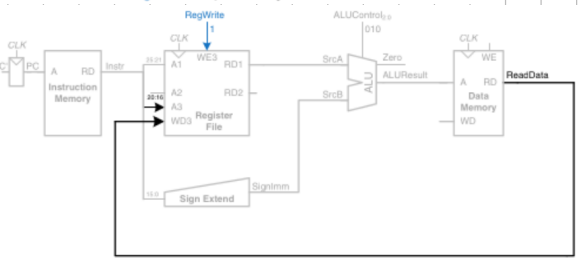


Figure 7.6 Write data back to register file

→ ReadData bus connected to WD3 of RF (port 3)

→ we added the control signal RegWrite for the write enable of port 3

→ this is asserted during lw because we want to write to RF (at next rising edge)

→ during the execute stage, we must compute PC' (next PC) → without branches, PC' = PC + 4 → we add an ALU to do this at next rising edge

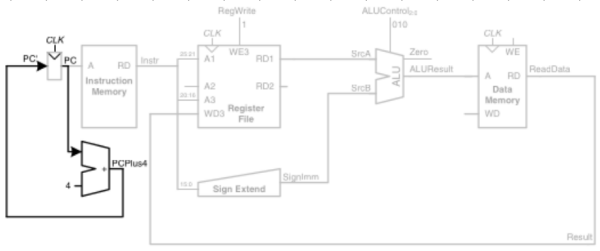


Figure 7.7 Determine address of next instruction for PC

→ next, we add support for sw → also uses base+offset

→ target reg. is in $Instr_{20:16}$ → connected to A2

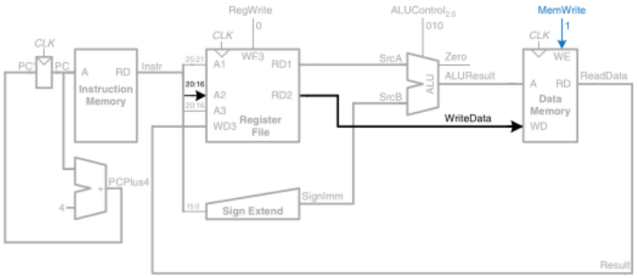


Figure 7.8 Write data to memory for sw instruction

→ added control signal MemWrite for the WE port of the data mem.

→ for sw: MemWrite = 1, ALUControl = 010, RegWrite = 0

→ next, we add support for R-type ins. (add, sub, and, or, slt)

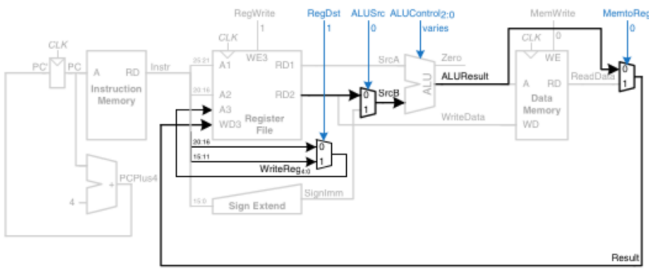


Figure 7.9 Datapath enhancements for R-type instruction

→ the RF reads 2 regs., the ALU operates

→ A MUX controlled by ALUSrc chooses SrcB from RD2 or SignImm → ALUSrc is 0 for R-types, 1 for I-types

→ R-types write to RF → we need a MUX to decide if the ALU result or data from mem. is pushed to RF → controlled by MemToReg (0 for R-type, 1 for lw)

→ The destination reg. is $Instr_{15:11}$ for R-types, so we need a MUX to choose WriteReg → controlled by RegDst, which is 1 for R-types (choose rd), 0 for lw (choose rt)

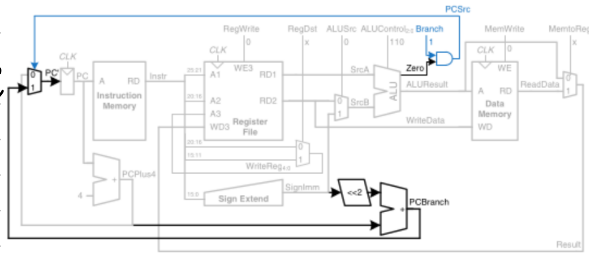


Figure 7.10 Datapath enhancements for beq instruction

→ now, add support for beq → we need to add the offset from imm to the PC

If the branch is taken → PC' = PC + 4 + SignImm × 4

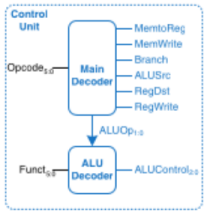
→ PCBranch: left shift by 2 bits

→ equality is checked by computing SrcA − SrcB → if Zero flag is up, regs. are equal

→ A MUX controlled by PCSrc chooses PC' between PCPlus4 and PCBranch → the signal is 1 if the ins. is a branch (Branch control signal) and Zero flag asserted

→ ALUControl is 110 (subtract), RegWrite and MemWrite are 0

Control Unit:



→ opcode and funct are used to compute control signals

→ we have 2 CL blocks, the main decoder for most outputs and the ALUOp signal, which the ALU decoder uses with funct to determine ALUControl

Table 7.1 *ALUOp* encoding

ALUOp	Meaning
00	add
01	subtract
10	look at funct field
11	n/a

Table 7.2 ALU decoder truth table

ALUOp	Funct	ALUControl
00	X	010 (add)
X1	X	110 (subtract)
1X	100000 (add)	010 (add)
1X	100010 (sub)	110 (subtract)
1X	100100 (and)	000 (and)
1X	100101 (or)	001 (or)
1X	101010 (slt)	111 (set less than)

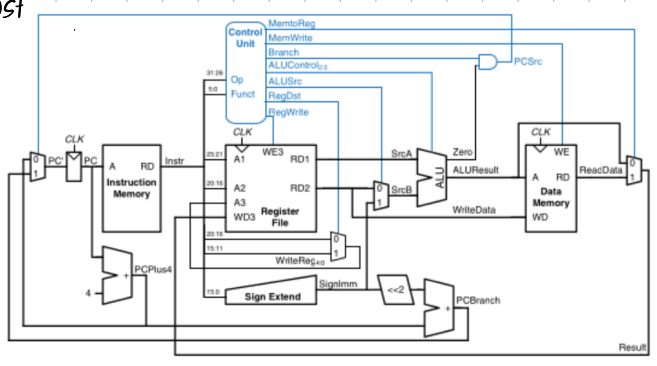


Figure 7.11 Complete single-cycle MIPS processor

→ On the main decoder, ALUOp is always 10 for R-types → funct decides

→ now, add support for addi → our datapath can handle this, we just have to add it to the decoder

→ Finally, we want to support j → needs datapath and TT expansion

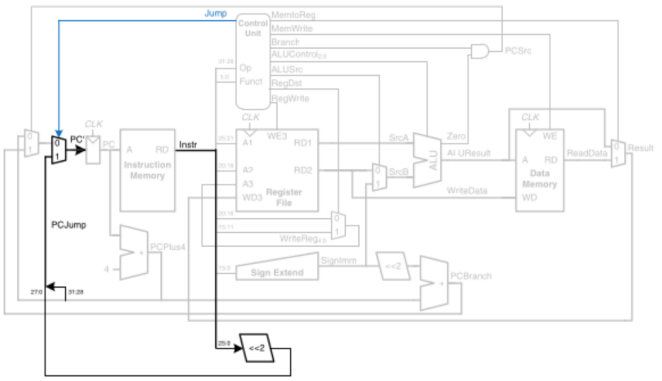


Figure 7.14 Single-cycle MIPS datapath enhanced to support the j instruction

Table 7.3 Main decoder truth table

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01

Table 7.4 Main decoder truth table enhanced to support addi

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	001000	1	0	1	0	0	0	00

Table 7.5 Main decoder truth table enhanced to support j

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	X	1	0	1	X	00	0
beq	000100	0	X	0	1	0	X	01	0
addi	001000	1	0	1	0	0	0	00	0
j	000010	0	X	X	X	0	X	XX	1

Performance Analysis:

Remember: Execution Time of an ins.: {CPI} x {Tc}

↳ Program: {#ins.} x {CPI} x {Tc}

→ If you only get frequency f in an exam, recall that $T_c = 1/f$

→ our current processor has a CPI of 1 (single cycle)

→ $T_c = t_{pcq\_PC} + t_{mem} + \max[t_{RFread}, t_{sext}] + t_{mux} + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$ (whole path)

→ ALU, mem., RF are usually slowest

⇒ $T_c = t_{pcq\_PC} + 2t_{mem} + t_{RFread} + 2t_{mux} + t_{ALU} + t_{RFsetup}$

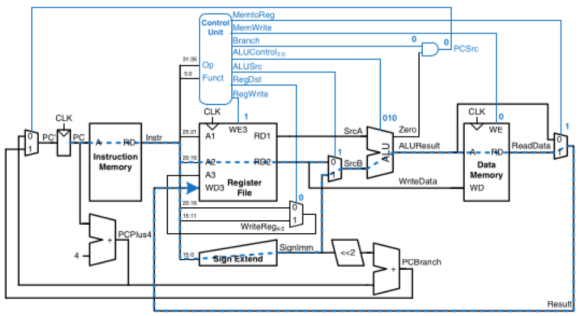


Figure 7.15 Critical path for lw instruction

→ often, questions like this come up:

Ben Bitdiddle is contemplating building the single-cycle MIPS processor in a 65 nm CMOS manufacturing process. He has determined that the logic elements have the delays given in Table 7.6. Help him compare the execution time for a program with 100 billion instructions.

Table 7.6 Delays of circuit elements

Element	Parameter	Delay (ps)
register clk-to-Q	t_{pcq}	30
register setup	t_{setup}	20
multiplexer	t_{mux}	25
ALU	t_{ALU}	200
memory read	t_{mem}	250
register file read	t_{RFread}	150
register file setup	$t_{RFsetup}$	20

→ Using the table, $T_c = 30 + 2(250) + 150 + 2(25) + 200 + 20 = 950$ ps

→ Exec. time = {$100 \cdot 10^9$ ins.} · {1 cycle/ins} · {$950 \cdot 10^{-12}$ s/cycle}

= $100 \cdot 950 \cdot 10^{-3}$ seconds = $95000 \cdot 10^{-3}$ seconds = 95 seconds (correct me if I'm BSing)

Multicycle CPU:

→ Ins. processing is broken into stages, state updates can be made during execution, arch. state updates at the end of execution

→ advantage: slowest stage determines cycle time → not bottlenecked by one ins.

→ all phases can take multiple clock cycles to complete

→ control signals needed in next cycle can be generated in current

→ latency of control processing can overlap with latency of datapath op. → more parallel

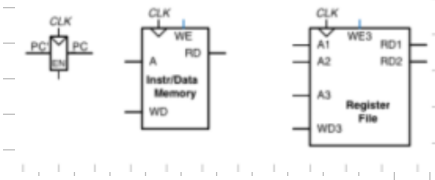
→ optimizing single-cycle CPU is harder because you cannot optimize for the common case, worst case optimization needed

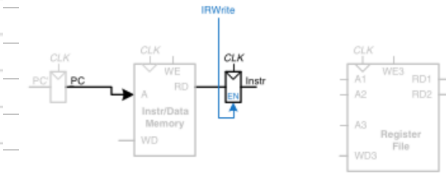
→ Design Principles: Critical Path design → decrease max. CL delay → breaks path into more cycles if needed

Bread and Butter (common case) → optimize for most used ins.

Balanced Design → balance ins./data flow through HW components, eliminate bottlenecks

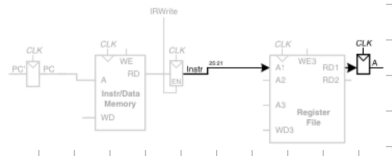
Actual Design: Major change: we use one mem. instead of 2 → read ins. in one cycle, read/write data in another





→ IR write signals when to update IR with new ins.

→ start with lw again



→ read source reg. containing base address

signed imm (here, offset) doesn't need its own reg. → because it will not change during an ins. exec.

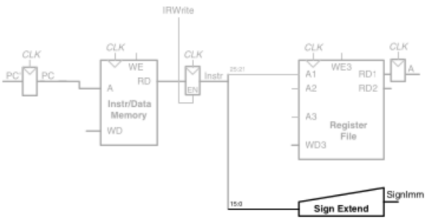


Figure 7.19 Sign-extend the immediate

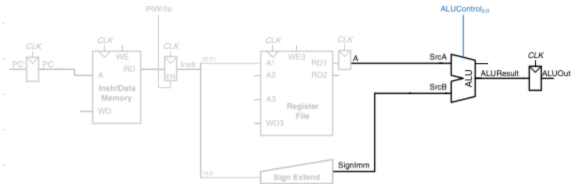


Figure 7.20 Add base address to offset

← ALUResult stored in nonarch. reg.

← ALUControl is as we know it from single-cycle design

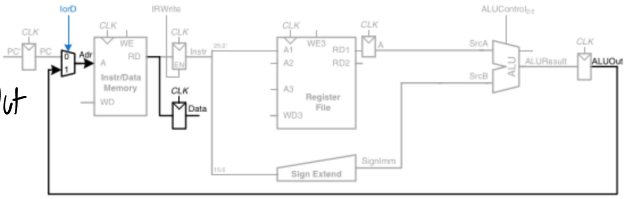


Figure 7.21 Load data from memory

→ a MUX has been added to decide mem. address between PC and ALUOut

→ controlled by IorD (ins. or data)

→ added nonarch. reg Data for data read from mem.

→ Data from Data written to reg. in Instr20:16

→ the same ALU is used to increment the PC (add PC and 4)

→ ALUSrcA controls a MUX that decides between PC or the reg. A

→ ALUSrcB has 4 options, for now we add only 4 and SignImm (PCWrite is a WE for the PC)

→ now, support sw:

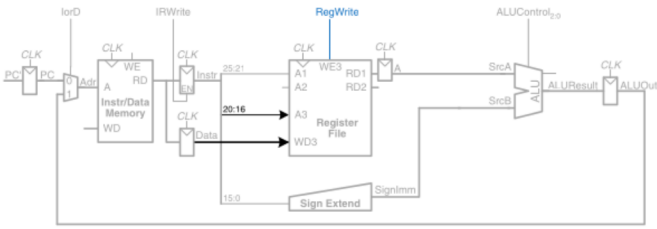


Figure 7.22 Write data back to register file

→ we connect AR to Instr20:16 and store it in B (nonarch) upon read

→ MemWrite control signal added to WE of memory

→ R-types:

→ we reveal another input for ALUSrcB → register B

→ ALUOut writeback into reg from Instr15:11 needs new MUXes

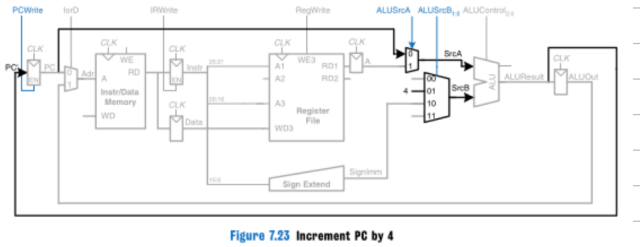


Figure 7.23 Increment PC by 4

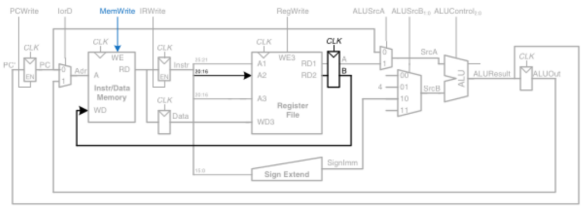


Figure 7.24 Enhanced datapath for sw instruction

→ MemToReg decides if WD3 comes from ALUOut (R-type) or Data (lw)

→ RegDst decides if destination comes from rt or rd field of ins.

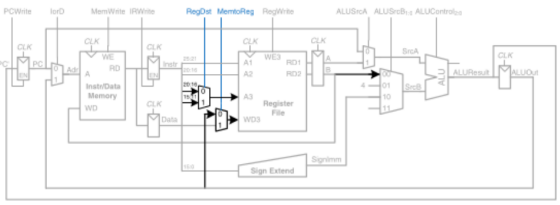


Figure 7.25 Enhanced datapath for R-type instructions

→ beq: we add the same mechanisms as in the single-cycle design:

→ if A/B are equal (Zero flag), we assert Branch MUX for beq and add PCWrite for later → both assert PCEn

→ PCSrc decides PC' between PC+4 and PC+ SignImm × 4 (branch taken)

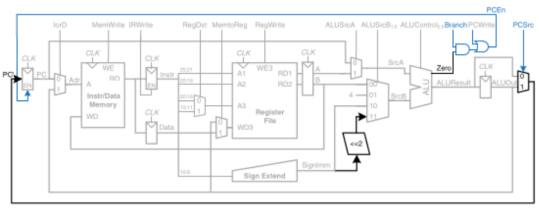
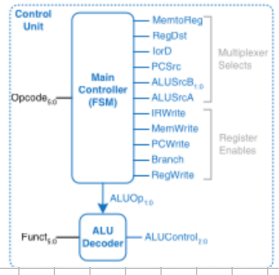


Figure 7.26 Enhanced datapath for beq instruction

Control: now, we use an FSM to control



→ separate write enables and MUX selects

→ Building FSM diagram: (start with fetch)

→ IorD = 0 → read memory, take addr. from PC

→ assert IRWrite to write ins. into IR

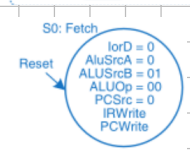
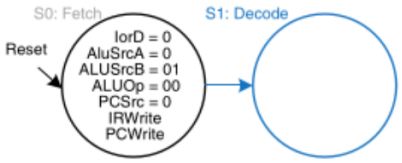


Figure 7.29 Fetch

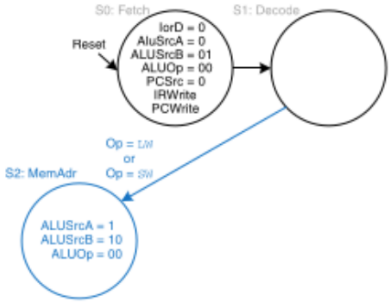
→ increment PC and use ALU s.t. we add PC + 4

→ decoding needs no control signals, but we wait a cycle for data to arrive



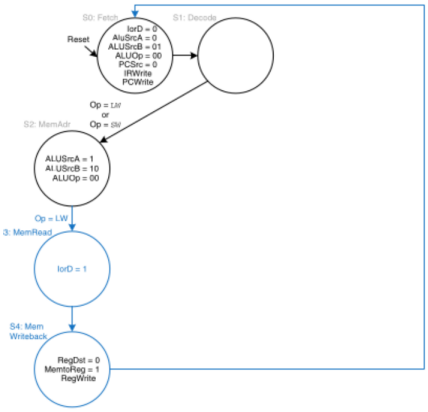
→ then, FSM proceeds to one of many states depending on opcode

→ if lw or sw, we need ALUSrcA = 1 to select A and ALUSrcB = 10 to select SignImm



→ ALUOp = 00, they are added → base + offset computation

→ if it's lw, proc. must read data from mem. and write to reg. file



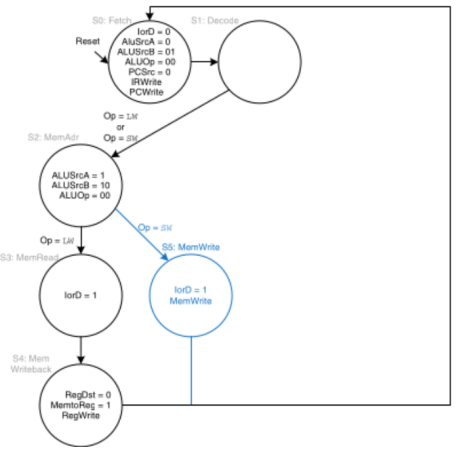
→ IorD = 1, selects ALUOut, which was saved in Data during S3

→ in S4, Data is written to RF → MemToReg = 1 selects Data, RegDst = 0 takes dest. from rt field

→ RegWrite is asserted → return to S0

→ from S2 → if we have sw, data from port 2 is written to Mem.

→ IorD = 1 → select address computed in S2, assert MemWrite → return to S0



R-type ins.:

→ calculate ALU result, write to RF

→ S6: select A/B registers and perform ALUOp from funct, store in ALUOut

→ S7: write ALUOut to RF ⇒ RegDst = 1 (dest. in rd field), MemToReg = 0

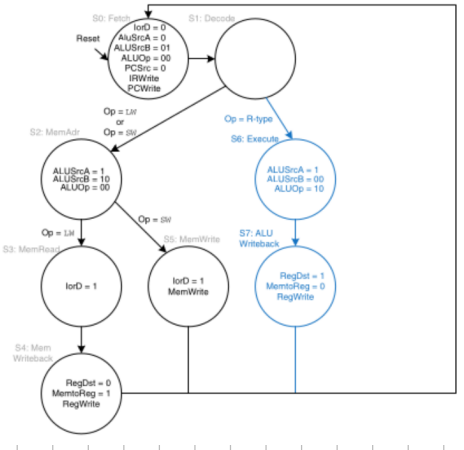
because write data comes from ALUOut, assert RegWrite

beq: (below)

→ proc. must calculate dest. address and compare sources to decide if branch taken

→ 2 uses of the ALU doesn't need 2 states, we can compute PC' during decode (ALU is idle then)

→ we speculate that the branch is taken ⇒ add PC + 4 to SignImm × 4 (ALUSrcA = 0, SrcB = 11, Op = 00)



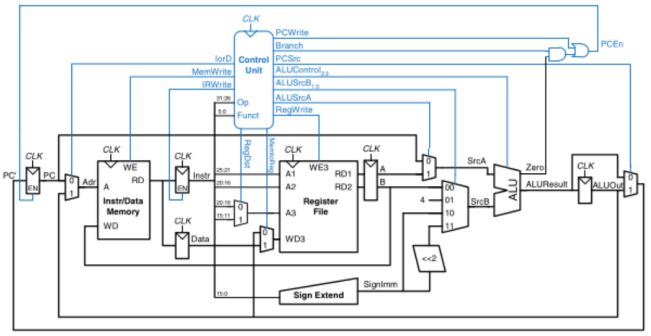


Figure 7.27 Complete multicycle MIPS processor

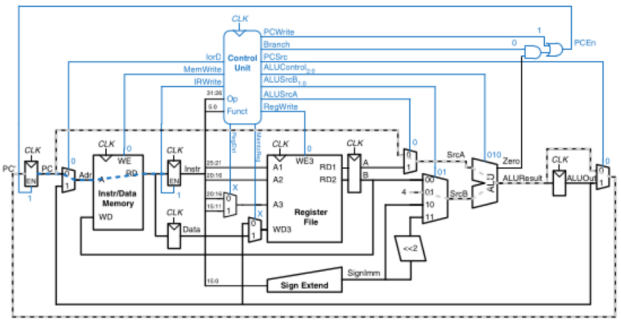


Figure 7.30 Data flow during the fetch step

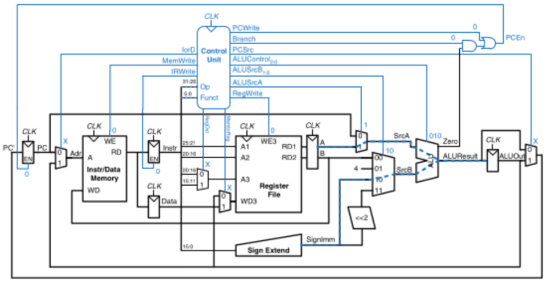
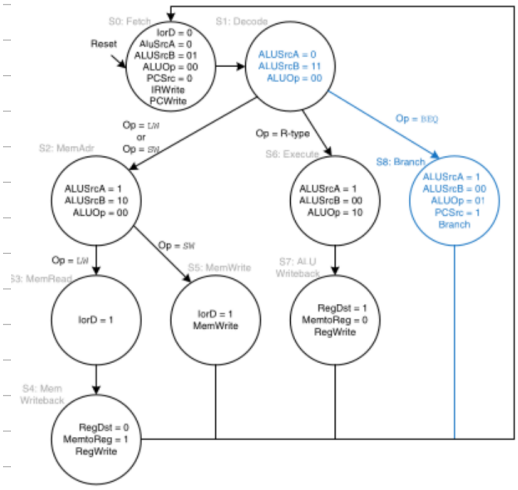


Figure 7.34 Data flow during memory address computation



→ computation does no harm if ins. wasn't branch

→ S8: proc. compares 2 regs. by subtracting, select PCSrc = 1 (from ALUOut), Branch = 1 ⇒ branch if result is 0

→ adding addi:

→ almost exact same to R-type

→ all we change is setting ALUOp (3) to add and source to SignImm

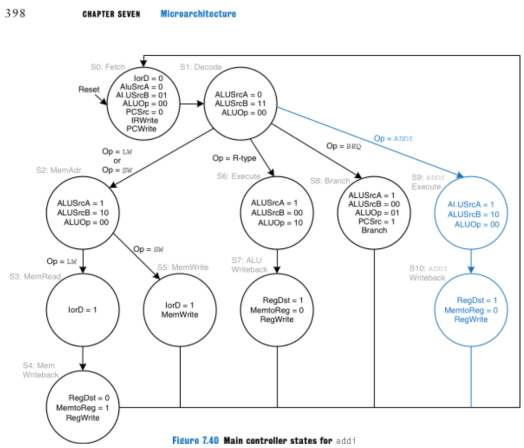


Figure 7.40 Main controller states for addi

Adding J: we must modify the datapath to support jump computation and extend PCSrc to take this address as an input

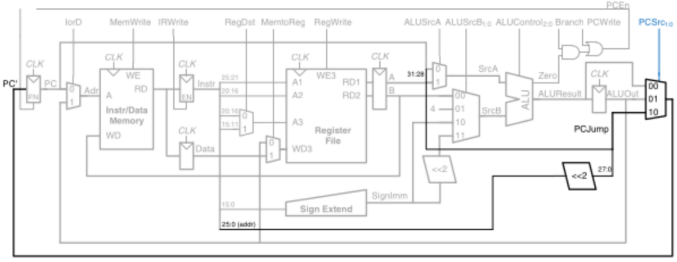


Figure 7.41 Multicycle MIPS datapath enhanced to support the j instruction

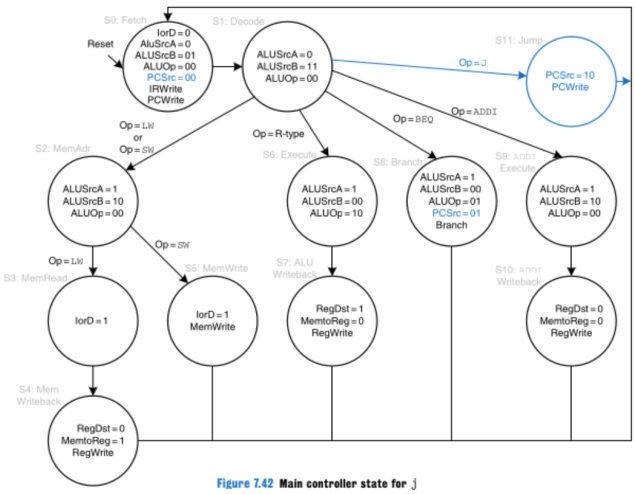


Figure 7.42 Main controller state for j

Performance Analysis: (I think this was taken out the exam pool, correct me if I'm wrong)

→ beq and j need 3 cycles, sw, addi, R-type need 4, lw needs 5

→ CPI depends on frequency of ins.

→ Example of calculating CPI:

Example 7.7 MULTICYCLE PROCESSOR CPI

The SPECINT2000 benchmark consists of approximately 25% loads, 10% stores, 11% branches, 2% jumps, and 52% R-type instructions.[3] Determine the average CPI for this benchmark.

→ CPI = 5·(0.25) + 3·(0.13) + 4·(0.62) = 1.25 + 0.39 + 2.48 = 4.12

→ Each cycle takes one ALU op., mem. access, or RF access → assume RF faster than mem., writing mem. faster than reading it:

⇒ $T_c = t_{pcq} + t_{mux} + \max(t_{ALU} + t_{mux}, t_{mem}) + t_{setup}$ → using the example from when we built the single-cycle processor, T_c would be 325ps

Pipelined Processor:

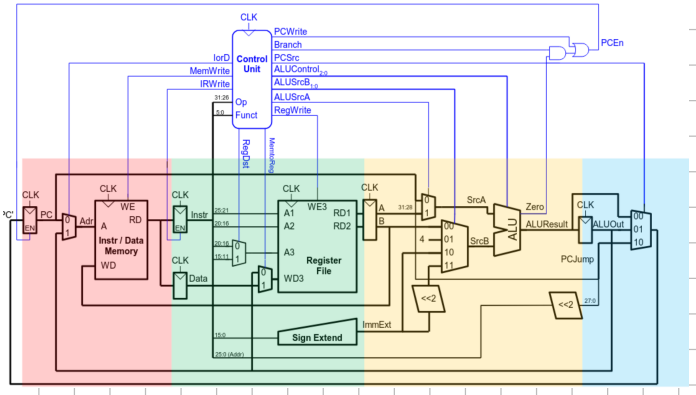
→ multi-cycle is a good start, but we can do much better in terms of concurrency

→ we have idle HW resources during different phases of ins. processing → example: fetch logic idle when ins. decoded

→ our goal is to complete more work per cycle

→ Idea: when an ins. is using some resources in its processing phase, process other ins. on idle resources

→ let's update our old model of ins. processing into 5 stages: 1. FETCH, 2. DECODE, 3. EXECUTE, 4. MEM (mem. operand fetch), 5. WB (writeback)

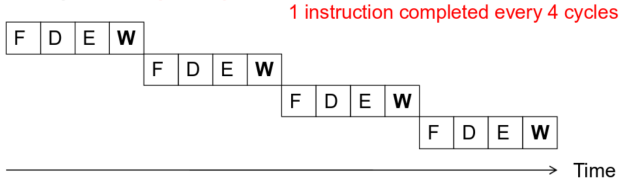


← basic idea of pipelining: systematically pipeline exec. of multiple ins.

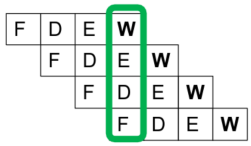
→ Idea: divide ins. processing cycle into 5 stages we discussed

→ aim to always process a different ins. in each stage

- Multi-cycle: 4 cycles per instruction
  1 instruction completed every 4 cycles

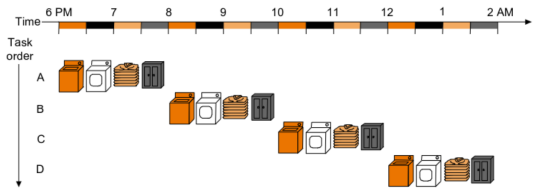


- Pipelined: 4 cycles per 4 instructions (steady state)
  1 instruction completed every 1 cycle



Is life always this beautiful?

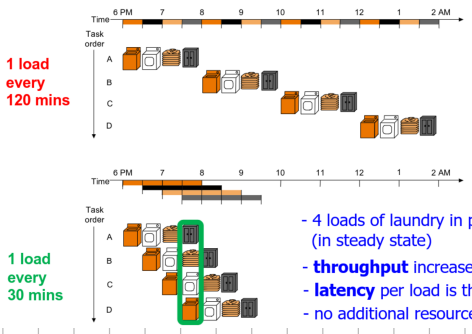
→ use laundry as an analogy:



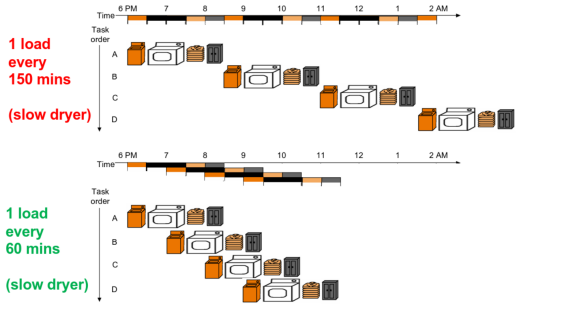
- "place one dirty load of clothes in the washer"
- "when the washer is finished, place the wet load in the dryer"
- "when the dryer is finished, take out the dry load and fold"
- "when folding is finished, put the clothes away"

→ sequential dependence between individual steps, but loads are independent

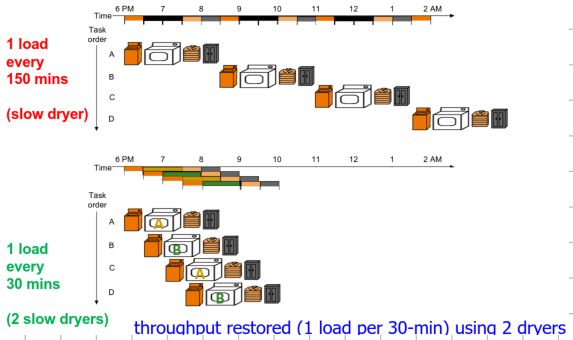
Ideally:



In practice:



→ here: solution is to get a new slow dryer



OK enough with the hook, let's build another CPU

→ our design will have 5 stages → exec 5 ins. simultaneously

→ clock frequency is 5 times faster than single-cycle CPU because each stage has 1/5 of the logic of the CPU

→ latency ideally unchanged, but 5x the throughput

→ timing diagram: ignores MUX/register delay

→ Single-cycle: serial latency of 950ps → throughput = $\frac{1 \text{ ins.}}{950 \text{ ps}}$ (1/latency) →

over 1 billion ins/second

→ pipeline: one stage takes 250ps (limited by slowest stage)

→ first fetch at 0ps, first decode/second fetch at 250ps, ...

→ latency of 1250ps → throughput = $\frac{5 \text{ ins.}}{1250 \text{ ps}} \rightarrow \frac{1 \text{ ins.}}{250 \text{ ps}}$ = 4 billion ins. per second

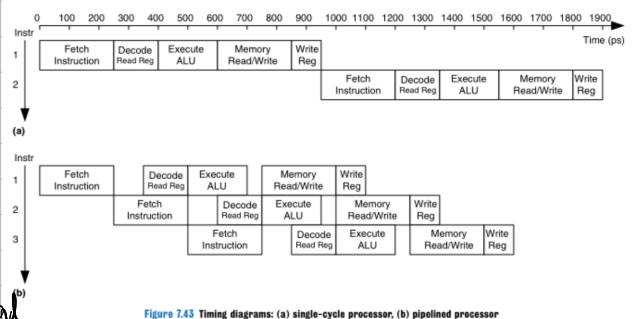


Figure 7.43 Timing diagrams: (a) single-cycle processor, (b) pipelined processor